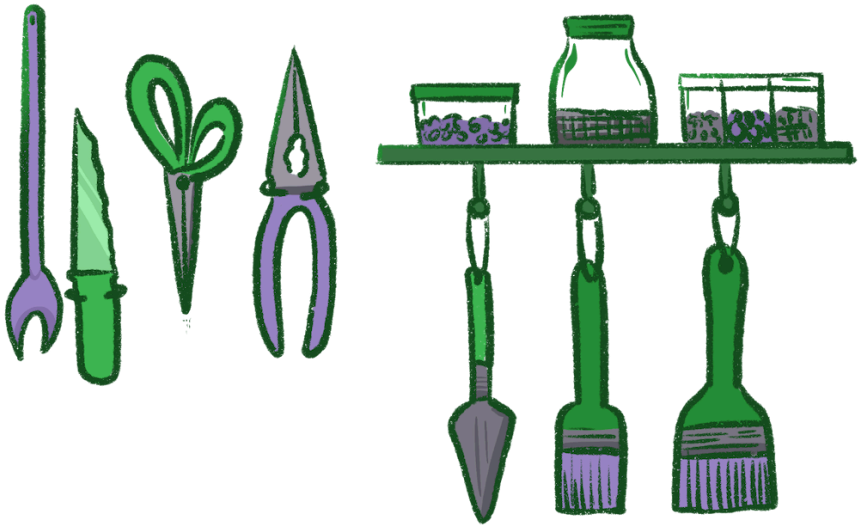
Sprites

Getting Started > Working with Sprites Copy page

Working with Sprites



After you've made it through the Quickstart, you've got a working Sprite and a basic idea of how to use it. This guide picks up from there: how to run commands, manage sessions, keep processes alive, and make sure your environment stays consistent over time. The first half covers everything you need to build and deploy real stuff. The rest is there when you're ready to go deeper.

Running Commands and Sessions

Sprites give you three main ways to interact:

sprite exec - One-off commands and automation

Run a single command, wait for it to finish, get the output. Perfect for scripts, package installs, or quick checks.

```
sprite exec ls -la
sprite exec npm install express
sprite exec -tty vim
```

- Blocks until the command completes
- Returns stdout/stderr
- Use for automation or scripting

sprite console - Interactive shell (like SSH)

Opens a full terminal session so you can explore, debug, or run multiple commands.

```
sprite console
# Inside:
# $ cd /home/sprite && ls -la && vim myfile.txt
```

- TTY enabled
- Stays open until you exit
- Use for manual work or debugging

Sessions - Keep things running

All TTY sessions are automatically detachable. Start a command, disconnect with `Ctrl+\`, and reattach later. Great for dev servers, long builds, or background processes.

```
sprite exec -tty npm run dev     # start a TTY session
# Press Ctrl+\ to detach

sprite sessions list             # list running sessions
sprite s ls                      # short form list sessions
sprite sessions attach <id>      # reattach to session
sprite sessions kill <id>        # kill session
```

Sprite Lifecycle: Idle Behavior and Persistence

When activity stops, Sprites immediately become `warm`. Over time they may transition to `cold`. `warm` Sprites resume quickly; `cold` Sprites take longer to wake. That means:

What Persists (and What Doesn't)

- ✓ **Filesystem persists**: All files, installed packages, git repos, databases—everything on disk stays intact
- ✗ **RAM doesn't persist**: Running processes stop, in-memory data is lost
- ✓ **Network config persists**: Open ports, URL settings, SSH access all remain configured

This means you can install dependencies once and they're there forever. But if you're running a web server, it'll need to restart when the Sprite wakes up.

Wake-up Behavior

Wake-up is fast:

- ~100–500ms for normal wakes
- 1–2s on cold starts

When a request hits your Sprite's URL, it wakes automatically. To make sure your web server is ready to handle that request, use **Services** — processes that auto-restart whenever your Sprite wakes up:

```
sprite-env services create my-server --cmd node --args server.js
```

Services survive hibernation. TTY sessions don't — they're great for interactive work and debugging, but any process started with `sprite exec` or `sprite console` stops when the Sprite sleeps.

Idle Detection

Your Sprite stays awake while there's activity, and sleeps when there isn't. Activity includes:

- Active exec/console commands
- Open TCP connections (like your app's URL)
- Running TTY sessions
- Active Services with open connections

Networking: URLs and Port Forwarding

Every Sprite gets a URL: `https://<name>.sprites.app`

HTTP Access

```
sprite url                      # see URL
sprite url update --auth public # make public
sprite url update --auth default # make private again
```

- Routes to port 8080 by default (or first HTTP port opened)
- Wakes the Sprite on request — pair with a Service so your server is ready to handle it
- Private by default (auth token required)

> ⚠ **Security note**: Public URLs expose your Sprite to the internet. Only use public mode for demos, webhooks, or non-sensitive work.

Port Forwarding

```
sprite proxy 5432           # access Sprite's port 5432 at localhost:5432
sprite proxy 3001:3000      # map local 3001 to remote 3000
sprite proxy 3000 8080 5432 # forward multiple ports
```

Use for database access, dev tools, or private ports. Press `Ctrl+C` to stop forwarding.

Port Conflicts

If a local port is already in use, you'll get an error. Solutions:

1. **Choose a different local port**: `sprite proxy 3001:3000` forwards local 3001 to remote 3000
2. **Stop the conflicting process**: Find what's using the port with `lsof -i :3000` (macOS/Linux)
3. **Kill the old proxy session**: If you have an old proxy still running, stop it first

Your Environment

Sprites run Ubuntu 25.10 with common tools preinstalled:

- **Languages**: Node.js, Python, Go, Ruby, Rust, Elixir, Java, Bun, Deno
- **AI/CLI Tools**: Claude CLI, Gemini CLI, OpenAI Codex, Cursor
- **Utilities**: Git, curl, wget, vim, and common dev tools

Filesystem Basics

- `/home/sprite/` — your home directory, put your stuff here
- `/home/sprite/.local/` — for local binaries and user-installed tools
- `/opt/` — good for standalone applications
- `/var/` — for databases and application state

Install packages like you would locally:

```
sprite exec pip install pandas numpy
sprite exec npm install -g typescript
sprite exec cargo install ripgrep
```

They persist across hibernation. No rebuilds needed.

> **Storage space**: Each Sprite has 100 GB of persistent storage. Check usage with:
>
> ```
> sprite exec df -h
> ```

Managing Sprites

Set Active Sprite

```
sprite use my-sprite
# Now all commands target this sprite
sprite exec echo "hello world"
```

List and Filter

```
sprite list
sprite list --prefix "dev-"
```

Destroy

```
sprite destroy -s my-sprite
```

> ⚠ **Destruction is irreversible!** All data is permanently deleted: files, packages, checkpoints. No undo.

Checkpoints

Snapshot your Sprite's filesystem so you can roll back later.

```
sprite checkpoint create
sprite checkpoint create --comment "before upgrade"
sprite checkpoint list
sprite restore <id>
```

Use before risky changes, upgrades, or experiments.

What gets saved:

✓ Entire filesystem (all files, installed packages, databases)

✓ File permissions and ownership

✗ Running processes (they stop during checkpoint creation)

✗ In-memory state

Good to know:

- Checkpoints count against your storage quota
- Restoring replaces the entire filesystem—changes since the checkpoint are lost

- Creation takes 10–30 seconds depending on data size

Optional: Going Deeper

These features are useful once you're comfortable.

Mounting Filesystem Locally

Use SSHFS to mount your Sprite and edit files with your local tools.

Sprites don't expose SSH directly—you'll need to install an SSH server on your Sprite and tunnel the connection through `sprite proxy`. This keeps your Sprite secure while still allowing local filesystem access.

Prepare an SSH server on your Sprite:

```
# Install OpenSSH
sudo apt install -y openssh-server

# Create a service to automatically start it
sprite-env services create sshd --cmd /usr/sbin/sshd
```

Install SSHFS on your local machine:

```
# macOS
brew install macfuse sshfs

# Ubuntu/Debian
sudo apt-get install sshfs

# Fedora/RHEL
sudo dnf install fuse-sshfs
```

Authorize your SSH public keys:

```
sprite exec mkdir -p .ssh
cat ~/.ssh/id_*.pub | sprite exec tee -a .ssh/authorized_keys
```

Add this helper to your shell config:

```
# Add to ~/.zshrc or ~/.bashrc
spritemount() {
  local sprite_name="$1"
  local mount_point="/tmp/sprite-mount"
  mkdir -p "$mount_point"
  sshfs -o reconnect,ServerAliveInterval=15,ServerAliveCountMax=3 \
    -o ProxyCommand="sprite proxy -s %h -W 22" "sprite@$sprite_name:" \
    "$mount_point"
  cd "$mount_point" || return 1
}

# Mount the sprite with "spritemount my-sprite"
```

Unmount when done:

```
umount /tmp/sprite-mount
# macOS may need: diskutil umount /tmp/sprite-mount
```

Common Error Scenarios

Connection errors:

- Check auth: `sprite org auth`
- Verify Sprite exists: `sprite list`
- Wait a moment and retry

Timeout errors:

- Be patient on first wake-up (1–2 seconds)
- Check if command actually needs that long

Sprite won't wake up:

- Verify it exists with `sprite list`
- Wait 30 seconds and retry
- Contact support if persistent

Storage full:

- Clean up files: `sprite exec bash -c "du -sh /home/sprite/*"`
- Delete old checkpoints

- Create a new Sprite for additional workloads

Quick debugging:

```
sprite exec ps aux      # running processes
sprite exec df -h       # disk space
sprite exec free -h     # memory usage
```

Sprites are meant to feel like your own Linux box in the sky—fast to wake, persistent when you need it, and flexible enough to run whatever weird stack you're building. As you get more comfortable, the advanced features are there when you need them.